

AI CLI Profile Manager 使用手册

适用版本: **0.3.6** (源码与安装目标; 发布、安装和 Live 证据须分别核对) 适用系统: Windows 11 x64、PowerShell 7 命令入口: **aicli**

AI CLI Profile Manager 是原生 Codex CLI、Claude Code 与 Open Interpreter 的启动和运维层。它负责保存 Profile、隔离 Provider 环境、管理可选代理、体检和连通测试; 聊天界面、历史会话、工具执行和上游账号仍由各 CLI 自己负责。

本手册是产品操作的唯一事实源。三套上游 CLI 的会话内命令见《[Codex、Claude Code 与 Open Interpreter CLI 中文手册](#)》。

1. 五分钟开始

1.1 安装前提

- Windows 11 x64。
- PowerShell 7, 命令为 **pwsh**。
- 至少安装一个目标 CLI: Codex、Claude Code 或当前官方 Rust Open Interpreter。
- 使用云端 API 的 Profile 需要对应 Provider 的 Key; 官方订阅 Profile 使用上游自己的登录。

1.2 安装本工具

0.3.6 是当前源码与安装目标; 源码提交、GitHub Release、已安装 payload 和 Live 回执不是同一层证据。从源码工作树安装时直接使用本节后面的 **scripts\Install.ps1**。使用正式发布版时, 从 [GitHub Releases](#) 下载同一版本的 ZIP 和 **.sha256.json**。下面的命令会先核对发布清单, 再解除这个已核对 ZIP 的 Internet 阻止标记; 不需要也不应该全局放宽 ExecutionPolicy:

```
$version = '<从 Releases 页面选择的已发布版本>'
$base = "https://github.com/wlyaaaaa/ai-cli-profile-manager/releases/download/v$version"
$download = Join-Path $HOME "Downloads\ai-cli-profile-manager-$version"
New-Item -ItemType Directory -Force -Path $download | Out-Null

$zip = Join-Path $download "ai-cli-profile-manager-$version-win-x64.zip"
$hashJson = Join-Path $download "ai-cli-profile-manager-$version.sha256.json"
Invoke-WebRequest "$base/$(Split-Path $zip -Leaf)" -OutFile $zip
Invoke-WebRequest "$base/$(Split-Path $hashJson -Leaf)" -OutFile $hashJson

$expected = (Get-Content -LiteralPath $hashJson -Raw | ConvertFrom-Json).sha256
$actual = (Get-FileHash -LiteralPath $zip -Algorithm SHA256).Hash
if ($actual -ne $expected) { throw 'SHA256 不匹配, 拒绝安装。' }

Unblock-File -LiteralPath $zip
$extract = Join-Path $download 'package'
Expand-Archive -LiteralPath $zip -DestinationPath $extract -Force
Set-Location $extract
pwsh -File .\scripts\Install.ps1
```

如果已经手动下载并核对文件, 从解压后的目录执行安装器即可:

```
pwsh -File .\scripts\Install.ps1
```

目标版本已经存在时，安装器默认拒绝覆盖。确认替换同一版本时使用：

```
pwsh -File .\scripts\Install.ps1 -Force
```

安装器会：

1. 把模块安装到当前用户的 PowerShell 模块目录，不要求管理员权限。
2. 创建 `%LOCALAPPDATA%\aicli\bin\aicli.cmd` 与 `aicli.ps1` 垫片，并尝试加入用户 `PATH`。
3. 尝试在当前用户 PowerShell 7 配置中加入模块自动导入块。
4. 先验证临时候选版本，再替换目标版本；失败时恢复原版本。
5. 升级到 `0.3.6` 时，先预检再隔离可证明由 AICLI 管理的 Qwen3.7 旧入口，同时保留新 06-08 exact Profile。

退役迁移不会把 Qwen3.7 自动改投 Qwen3.8，也不读取、复制、移动或删除 SecretRef/密钥。身份、marker、body/state 哈希与内容寻址都闭合的旧用户 Profile、Codex TOML/catalog 和旧模块版本会移入可恢复目录

`%LOCALAPPDATA%\AiCliProfileManager\retirement\qwen37-v1`。任何未知、用户改写或 reparse 项都会在任何安装变更前失败关闭；先审计该路径，不要盲目删除。

新开一个 PowerShell 7 窗口后验证：

```
Get-Command aicli  
aicli version
```

当前窗口还找不到 `aicli` 时：

```
$env:Path = "$env:LOCALAPPDATA\aicli\bin;$env:Path"  
Import-Module AiCliProfileManager  
aicli version
```

开发者也可以不安装，直接在仓库根目录运行：

```
pwsh -File .\bin\aicli.ps1 version  
pwsh -File .\bin\aicli.ps1 doctor
```

1.3 第一次设置

```
aicli setup
```

`setup` 会检查 Windows、PowerShell、目标 CLI、Profile、Ollama 和可选代理，并让你选择要配置的方向。静态检查不会让模型生成内容；配置第三方云端 Profile 时才会要求无回显输入 API Key。

1.4 开始第一段对话

在项目根目录运行：

```
aicli start codex-official
# 或
aicli start claude-official
```

也可以显式指定含空格或中文的目录：

```
aicli start codex-official --project "C:\Work\Demo App"
aicli start claude-official --project "D:\项目\演示"
```

命令会立即启动一个新的原生 CLI 进程。父终端的 Provider 环境变量不会被永久修改。

1.5 工作区日常用法（精确 Codex Profile）

在受信任的项目根目录用一条命令进入所需的精确 Codex CLI：

```
cd E:\你的项目
# 例：cd C:\Work\my-project

aicli start codex-qwen3-7-max-paygo --project (Get-Location)
# 或 Qwen3.8 Max
aicli start codex-qwen3-8-max-paygo --project (Get-Location)
# 或 DeepSeek V4 Flash 0731 / Pro 0813
aicli start codex-deepseek --project (Get-Location)
aicli start codex-deepseek-v4-pro --project (Get-Location)
```

说明：

- `--project` 必须是已经确认可信的工作区；Profile 自己封闭 Provider、模型、Responses wire、MAX 与 SecretRef。
- Qwen3.7 只恢复 `codex-qwen3-7-max-paygo` → `qwen3.7-max-2026-06-08`；通用 alias、其他快照、Plus、旧 Profile、导入和原生 `--model` / fallback 仍失败关闭。
- DeepSeek Codex 只保留下列两个 exact 身份：

模型身份	Profile
deepseek-v4-flash / DeepSeek-V4-Flash-0731	codex-deepseek
deepseek-v4-pro / DeepSeek-V4-Pro-0813	codex-deepseek-v4-pro

1.6 安装或升级 aicli 之后必须用新会话

`Install.ps1 -Force` 或手动覆盖模块目录后，已经打开的 PowerShell 窗口仍可能加载旧模块。会出现「别人已修好、你这边还在报 engine / 旧行为」的假象。

任选其一：

```
# 推荐：关掉终端，新开 pwsh 再运行 aicli
# 或在当前窗口强制重载：
Remove-Module AiCliProfileManager -Force -ErrorAction SilentlyContinue
Import-Module AiCliProfileManager -Force
aicli version
```

2. 先理解三个对象

对象	保存什么	是否包含秘密
Provider 模板	随产品发布的引擎、端点、模型和能力声明	否
用户 Profile	选择的模板、地域、模型、项目偏好和秘密引用	只保存引用
启动计划	本次可执行文件、工作目录、参数和子进程环境	运行时短暂持有必要秘密

`aicli start` 每次根据模板和用户 Profile 重新生成启动计划，因此更新或重新安装上游 CLI 后，不需要把 Key 写进全局环境变量。Profile 切换只对**新进程**生效，不能在已经打开的会话中途热切换 Provider。

`aicli run` 是供上层程序使用的 Codex harness，不等于交互式 `start`：所有当前和未来 Codex 模型固定使用原生 `danger-full-access` 与 `approvalPolicy=never`。规则按 `engine=codex` 生效，因此未来新增 Profile 自动继承；显式请求 `read-only` / `workspace-write` 会在模型调用前失败关闭。运行还必须回读 actual model、modelProvider 与 `dangerFullAccess` 权限身份，并拒绝任何模型重路由。只把受信任工作区交给该入口。

同一通用规则默认注册受管 `public_web_search`：仅访问固定 `https://cn.bing.com/search` RSS，拒绝重定向、Cookie、模型指定 endpoint/Header/Key，并把结果标作不可信公共文本。Windows 系统代理只用于固定 HTTPS 出口且不提供默认凭据。事件和回执仅记录搜索生命周期、provider 与次数，不记录查询/结果正文；本次不需要网络时使用 `--no-web-search`，权限仍是 `danger-full-access`。

3. Profile 管理

3.1 命令总览

```
aicli profile list
aicli profile list --available
aicli profile list --available --json
aicli profile show codex-deepseek
aicli profile configure codex-deepseek
aicli profile configure codex-deepseek-v4-pro --reuse-secret-from codex-deepseek
aicli profile set-default codex-official
aicli profile remove qwen-work
```

命令	作用	生效时间
<code>profile list</code>	显示可直接启动或已经配置的 Profile	立即
<code>profile list --available</code>	显示全部公开模板和当前状态	立即
<code>profile show</code>	脱敏显示 Provider、端点、模型、密钥是否存在和数据去向	立即
<code>profile configure</code>	保存用户 Profile；需要时无回显录入 Key并用 DPAPI 加密	下次启动
<code>profile set-default</code>	设置无参数入口优先使用的 Profile	下次无参数运行
<code>profile remove</code>	删除用户实例；不删除内置模板	立即

删除型命令会先确认。使用 `--yes` 可跳过交互确认。删除最后一个引用某密钥的用户 Profile 时，也会删除本工具保存的对应 DPAPI 密钥文件；不会删除上游官方登录。

3.2 当前公开 Profile

引擎	场景	Profile
Codex	ChatGPT/OpenAI 官方登录	codex-official
Codex	Qwen3.7 Max 06-08 Workspace 按量 Responses	codex-qwen3-7-max-paygo
Codex	Qwen3.8 Max Workspace 按量 Responses	codex-qwen3-8-max-paygo
Codex	DeepSeek V4 Flash 0731 / Pro 0813 Responses	codex-deepseek 、 codex-deepseek-v4-pro
Codex	本机精确 main / review	codex-ollama-main 、 codex-ollama-review
Claude Code	Claude 官方登录	claude-official
Claude Code	DeepSeek V4 Flash	claude-deepseek
Claude Code	本机 Ollama	claude-ollama
Claude Code	自定义 Anthropic Messages 兼容端点	claude-custom
Claude Code	ChatGPT 第三方本地代理	claude-chatgpt-ccp 、 claudex-chatgpt-cliproxy
Open Interpreter	DeepSeek V4 Flash Chat	oi-deepseek
Open Interpreter	本机 Ollama	oi-ollama

codex-deepseek 精确固定 API alias deepseek-v4-flash / 版本 DeepSeek-V4-Flash-0731 , codex-deepseek-v4-pro 精确固定 alias deepseek-v4-pro / 版本 DeepSeek-V4-Pro-0813 。两者均使用官方 Codex integration 的 Responses wire、1M context、low / high / max , 用户默认 max , 并拒绝模型、Provider 与 fallback 覆盖。动态变化以 DeepSeek Change Log 为准; 非 Codex 的 Claude Code / Open Interpreter DeepSeek 模板仍保持 Flash-only。

两个 Qwen exact Profile 都只接受北京百炼 Workspace 按量 Responses endpoint, 并固定 983616 context、95% 有效窗口和 262144 token 自动压缩阈值。codex-qwen3-7-max-paygo 只固定 qwen3.7-max-2026-06-08 ; codex-qwen3-8-max-paygo 只固定 qwen3.8-max 。用户 max 均映射 effective= xhigh 。Qwen3.7 的通用 alias、其他快照、preview、Plus、原有 Codex/Claude/OI 入口与模型覆盖继续失败关闭。

Claude Code 2.1.193+ 的 DeepSeek Profile 按最终 --model 精确注入 1000000 token 模型窗口。AICLI 不设置会提前压缩的 CLAUDE_AUTOCOMPACT_PCT_OVERRIDE , 也不默认禁用自动/手动压缩; 未知或自定义模型不猜测。

本机 claude-ollama-main 同样按 qwen-main-v1=262144 注入 MAX/AUTO; codex-ollama-main 使用独立的 262144 受管目录, 不再采用 Codex 未知模型 272K 回退。任何第三方 Claude 模型未命中受管元数据时, AICLI 会清除父进程遗留的 MAX/AUTO、提前压缩百分比和两个禁压缩变量, 再交给 Claude Code 的保守默认, 避免把上一模型的窗口误套到未来模型。

Codex 的 DeepSeek Key 与其他云端 Profile 一样由 Windows CurrentUser DPAPI 保存。受管 Codex TOML 只写 env_key , 不会复制官方手工示例中的明文 experimental_bearer_token 或 preferred_auth_method 。Qwen Code 0.21 与 OpenCode 1.18.8 虽有上游原生 DeepSeek 接入, 但 AICLI 当前只为它们提供禁网的 machine-only 沙箱, 且尚无隔离真实 Key 的远程 egress relay; 因此不提供这两类 DeepSeek Profile。

Ollama 公共模板使用默认地址 127.0.0.1:11434 , 不再包含某台机器的私有端口或模型别名。使用前先确认服务和模板所列模型实际存在:

```
ollama list
aicli doctor codex-ollama
```

3.3 配置第三方 API

以 Qwen3.7/3.8 Max 与 DeepSeek 为例:

```
aicli profile configure codex-qwen3-7-max-paygo --reuse-secret-from codex-qwen3-8-max-paygo
aicli profile configure codex-qwen3-8-max-paygo
aicli profile configure codex-deepseek
aicli profile configure codex-deepseek-v4-pro --reuse-secret-from codex-deepseek
aicli profile configure claude-deepseek
aicli profile configure oi-deepseek
```

配置过程中会先显示引擎、Provider、套餐和数据去向，再无回显读取 Key。秘密使用 Windows DPAPI CurrentUser 保存，不写入 Git、日志、`profile show`、`native` 或 `eject`。

3.4 启动、查看与导出

```
aicli start claude-deepseek --project "C:\Work\Project"
aicli native claude-deepseek
aicli eject claude-deepseek --output .\export-claude-deepseek
```

- `start` 启动真实上游 CLI。
- `native` 脱敏显示可执行文件、配置、环境差异和数据去向，不执行模型请求。
- `eject` 导出不含 Key/OAuth 的独立配方；目标目录已存在时应换一个新目录。

原生参数必须放在双横线之后：

```
aicli start claude-official -- --effort high
aicli start claude-deepseek -- --permission-mode acceptEdits
aicli start codex-official -- --model gpt-5.6-terra
```

与本工具启动计划冲突的 Provider、认证、`--profile`、`--settings` 或 `-c` 覆盖会被拒绝。不要用透传参数绕过 Profile 的安全边界。

3.5 从 OpenClaw 安全导入（可选）

如果 OpenClaw 已经保存了 DeepSeek 配置，可以用发行包里的导入脚本减少重复录入。它默认只预览，不修改 Profile，也不显示 Key；Qwen3.7 配置会被明确忽略：

```
pwsh -File .\scripts\Import-FromOpenClaw.ps1
```

配置文件不在默认的 `%USERPROFILE%\openclaw\openclaw.json` 时，显式指定路径：

```
pwsh -File .\scripts\Import-FromOpenClaw.ps1 -OpenClawJson "D:\配置\openclaw.json"
```

确认预览内容后再导入：

```
pwsh -File .\scripts\Import-FromOpenClaw.ps1 -Apply
```

已有同名 Profile 时默认跳过；只有明确要替换时才使用：

```
pwsh -File .\scripts\Import-FromOpenClaw.ps1 -Apply -Force
```

脚本只接受能由 HTTPS 主机名证明身份的 DeepSeek 配置，并要求主机为 `api.deepseek.com`。导入会生成 `codex-deepseek`、`codex-deepseek-v4-pro`、`claude-deepseek`、`oi-deepseek`；不会创建任何 Qwen3.7、Qwen Code 或 OpenCode 路径。未知/OpenAI Base URL 会被拒绝，避免把某家的 Key 误送到另一家。导入的 Key 立即使用 Windows CurrentUser DPAPI 保存；输出、Profile JSON 和日志中都不出现明文。导入后运行 `aicli doctor <Profile ID>`，需要真实连通证据时再显式执行 Live Test。

4. Doctor 与 Live Test

4.1 Doctor：静态检查，不生成模型内容

```
aicli doctor
aicli doctor codex-deepseek
aicli doctor codex-deepseek --json
```

Doctor 检查 CLI、Profile、秘密引用、端点、有效配置层、代理、端口和本机服务。它不发送模型生成请求，不应消耗模型额度。

状态	含义
通过	单个检查项成功
可用	要求的真实验证和必要能力均有当前证据
可用但有限制	静态就绪但未完成全部 Live Test，或存在已知限制
不可用	关键依赖、配置或测试失败

找不到可选 CLI 不应否定其他引擎；请按 check ID 和“下一步”处理目标 Profile 的问题。

4.2 Live Test：真实请求，可能消耗额度

```
aicli test codex-deepseek --live --level text
aicli test claude-deepseek --live --level text
aicli test claude-deepseek --live --level text --yes
aicli test claude-deepseek --live --level all --yes --json
```

`--live` 必须显式给出；没有 `--yes` 时会再次确认。文本测试在随机临时空目录中运行，通过目标 CLI 发出最小请求，并要求正常退出、最终模型正文严格等于 `PONG`，且观测到的工具调用数为 0。Codex 路径仍固定 `danger-full-access`；只要 app-server 报告首个工具事件就终止并判失败，但这不是执行前工具禁用，工具仍可能在事件被观测前产生本机副作用，只应在明确授权时执行。提示和回复正文不写日志。

层级	含义
text	验证目标 CLI、Provider、模型和最小文本链路
tool	只验证隔离的单用途 nonce 工具链
all	依次验证两层

无法证明工具隔离时，工具层会跳过并显示“可用但有限制”，不会冒充完整文件或 Shell 编程能力已经通过。

当前哪些 Profile 已完成文本验收、哪些仍待验收，以《兼容性与最终验收状态》为准；成功记录也不能跨模型、端点或公开默认配置外推。

5. 当前官方 Rust Open Interpreter

本工具只支持当前官方 Rust Open Interpreter `0.0.21` 或更高版本，不支持旧 Python `open-interpreter 0.4.x`。

5.1 安装和验证

```
irm https://www.openinterpreter.com/install.ps1 | iex
interpreter --version
aicli doctor
```

版本输出必须类似 `interpreter 0.0.21` 或更高。若检测到 `Open Interpreter 0.4.x`，Doctor 会把它识别为不受支持的旧 Python 版本，并给出当前官方安装命令。

5.2 配置和启动

```
aicli profile configure oi-deepseek
aicli start oi-deepseek

aicli start oi-ollama
```

aicli 通过 Rust CLI 的临时 `-c` 配置层写入 Provider、模型、`base_url` 和 `wire_api`。云端 Key 只进入目标子进程环境变量，且明确从 OI Shell 工具环境中排除；不会放进命令行参数。默认不启用跳过审批、跳过沙箱或旧版 `auto_run` 参数。Open Interpreter 仍具备本机代码执行能力。只在信任的项目目录和 Provider 中使用，并在 OI 自己的 `/permissions` 中确认权限。

官方资料：[安装](#)、[快速开始](#)、[配置](#)、[Provider](#)。

6. Claude Code 权限与第三方模型

aicli 默认不替用户设置 Claude permission mode。底栏 `manual mode on` 是 Claude Code 的默认权限档，不是启动失败。会话内按 `Shift+Tab` 通常在以下模式间循环：

```
manual → acceptEdits → plan
```

查看工具规则：

```
/permissions
```

启动时指定更松的日常写代码模式：

```
aicli start claude-deepseek -- --permission-mode acceptEdits
```

完全绕过权限只适合外部已经隔离、没有不可信网络或文件的环境：

```
aicli start claude-deepseek -- --permission-mode bypassPermissions --dangerously-skip-permissions
```

PowerShell 有时会吞掉参数分隔用的 `--`，导致 `--permission-mode` 等并未传给 Claude。需要透传时优先：

```
pwsh -NoProfile -File $env:LOCALAPPDATA\aicli\bin\aicli.ps1 --% start claude-deepseek -- --permission-mode acceptEdits
# 或开发仓库:
pwsh -NoProfile -File <安装目录>\bin\aicli.ps1 --% start claude-deepseek -- --permission-mode acceptEdits
```

`bypassPermissions` 必须在新进程启动时启用。第三方模型通常不满足 Claude `auto` 模式的模型或账号条件；这不是 aicli 的故障。

Claude 官方登录与 `ANTHROPIC_API_KEY` 同时存在时，上游可能显示双认证或 `connectors disabled` 提示。使用第三方 API Profile 时以该 Profile 显示的数据去向为准；使用 `claude-official` 时，先运行 Doctor 检查会抢占官方路径的设置层。

6.1 第三方模型的上下文连续性

原生 ChatGPT + Codex 是基准，保持上游默认。DeepSeek/千问经 Codex 或 Claude Code 时，客户端压缩是可能丢文件细节、数字和未完成分支的摘要；不要为了“省上下文”主动 `/compact`。一个会话只做一个内聚里程碑，在自然边界把目标、约束、已改文件、决定、测试、阻塞和下一步写入项目已有 plan/progress；接近窗口时优先 fresh session 或拆任务。压缩后重新读取适用的 `AGENTS.md` / `CLAUDE.md`、当前 `SKILL.md`、状态文档以及 `git status` / `git diff`。

本机 `qwen-main-v1` 在 Codex、Claude Code 与 OpenCode 中统一声明 262144 context。`opencode-ollama-main` 的一次性配置另固定 8192 output、20000 reserved，关闭 tool-output pruning 并保留最近 4 轮/16384 token；

`agent.build.steps` 只算上游软收尾，不冒充 AICLI 硬工具调用门。OpenCode checkpoint 随 run 清理，跨 run 连续性必须靠项目状态文件。远程 DeepSeek/千问 OpenCode 仍因 key-isolated egress relay 缺口不开放。

第三方模型（如千问）下，Claude 会话结束页的 **Total cost: \$... (costs may be inaccurate due to usage of unknown models)** 不是 阿里云百炼账单。token 量级可能接近真实调用；美元金额多半按 Claude 内置未知模型单价估算，**往往偏高**。真费用以百炼「模型监控 / 账单」为准（调用后约一小时可查）。详见《CLI 中文手册》用量说明。

7. ChatGPT 双代理（可选）

两个通道都是第三方本地代理，不是 OpenAI 或 Anthropic 官方功能，也不自动串联或故障转移。

ID	上游	角色	首选端口
ccp	raine/claude-code-proxy	Claude Code 到 ChatGPT 的专用转换器	43197
cliproxy	router-for-me/CLIProxyAPI	多协议备用通道	43198

0.1.0 未完成两个代理的交互 OAuth 和端到端 Live Test；它们只能标为“可用但有限制”，不能由安装成功推断为订阅链路可用。

完整流程：

```
aicli proxy ccp install
aicli proxy ccp configure --auto-port
aicli proxy ccp login
# 无法打开本机浏览器时:
aicli proxy ccp login device
aicli proxy ccp start
aicli proxy ccp status
aicli start claude-chatgpt-ccp
```

将 `ccp` 换成 `cliproxy` 可使用另一通道。CLIProxyAPI 登录还可显式选择 `codex`、`claude` 或 `device`：

```
aicli proxy cliproxy login codex
aicli proxy cliproxy login device
```

安全边界：

- 下载的 Windows artifact 必须命中产品批准的精确 SHA256，否则拒绝安装或更新。
- 受管代理只允许监听 `127.0.0.1`；发现 wildcard、局域网或公网监听会停止本次受管进程。
- 首选端口不可用时在受控候选池中重试，不杀未知占用进程。
- 本工具不读取 OAuth token 正文。
- `logout --purge-local-auth` 只删除本项目隔离的本地认证目录，不等于远程撤销授权。

退出与停止：

```
aicli proxy ccp stop
aicli proxy ccp logout
aicli proxy ccp logout --purge-local-auth --yes
```

远程授权请另外在对应账号的安全或已授权应用页面确认撤销。

8. 更新与修复

8.1 只读检查

```
aicli update check
aicli update check codex --json
aicli update guide codex
aicli update guide claude
aicli update guide ollama
aicli update guide interpreter
aicli update guide self
```

`update check` 和 `update guide` 只检查来源、版本并打印同渠道指引，不静默升级 Codex、Claude Code、Ollama 或本工具。识别来源后，应继续使用原安装渠道，避免 npm、WinGet 和原生安装器互相覆盖。

从任何旧 AICLI 版本升级到 `0.3.6` 时，应使用同一发行包内的 `scripts\Install.ps1`；该安装器会完成旧 Qwen3.7 可验证遗留入口的可恢复隔离，并保留新 06-08 exact Profile。如果预检报告未知或篡改项，安装在写入新版本前中止；请先备份并人工审计，不要绕过门禁。

Open Interpreter Rust 可按上游支持使用：

```
interpreter update
interpreter --version
```

受管代理版本检查：

```
aicli proxy ccp update-check
```

`0.1.0` 只对未安装代理执行经过固定 SHA256 的首次 `install`。对已经安装的代理，`update` 会明确拒绝且不会替换当前版本；先用 `update-check` 查看批准状态，等待后续具备运行健康回滚的版本再做受管升级。

8.2 通用修复顺序

1. `aicli update check` 确认实际来源和版本。
2. 使用同一渠道更新目标 CLI。
3. 运行目标 CLI 的 `--version`。
4. 运行 `aicli doctor <Profile ID>`。
5. 静态通过后，再按需运行显式 Live Test。

9. 卸载、恢复与数据

9.1 普通卸载

```
aicli uninstall
# 非交互确认
aicli uninstall --yes
```

普通卸载会停止本工具识别的受管代理，并完整移除当前用户模块、`%LOCALAPPDATA%\aicli\bin` 命令垫片、安装器加入的用户 `PATH` 项，以及带产品标记的 PowerShell Profile 自动导入块；不需要再手动清理这些安装集成。

普通卸载默认保留用户 Profile、DPAPI 密钥、代理本地数据和导出物，便于以后重装恢复。它不会卸载 Codex、Claude Code、Open Interpreter、Ollama、本地模型，也不会删除上游官方登录。

9.2 彻底清理本工具数据

先处理代理登录：

```
aicli proxy ccp logout
aicli proxy cliproxy logout
```

然后：

```
aicli uninstall --purge-user-data --yes
```

`--purge-user-data` 删除本工具的 Profile、DPAPI 密钥、日志、缓存、代理二进制和代理本地认证目录。它不代表远程 OAuth 已撤销。

9.3 数据位置与去向

内容	默认位置
Profile、书签和非秘密设置	<code>%APPDATA%\AiCliProfileManager</code>
DPAPI 密文、代理、状态、缓存和日志	<code>%LOCALAPPDATA%\AiCliProfileManager</code>
命令垫片	<code>%LOCALAPPDATA%\aicli\bin</code>

工具自身默认无遥测。使用某个 Profile 时，提示、项目上下文和相关数据会发送到 `profile show` 显示的 Provider 或本地代理；Ollama 在本机处理，Open Interpreter 还可能在本机执行代码。Live Test 会联系目标 Provider，并可能消耗额度，但不保存提示或回复正文。

10. 按症状排障

找不到 `aicli`

```
$env:Path = "$env:LOCALAPPDATA\aicli\bin;$env:Path"
Import-Module AiCliProfileManager
aicli version
```

仍失败时，新开 PowerShell 7，再用安装包中的 `scripts\Install.ps1 -Force` 修复。

升级后行为仍像旧版 / 报奇怪的 `engine` 属性错误

当前窗口可能仍在用内存中的旧模块：

```
Remove-Module AiCliProfileManager -Force -ErrorAction SilentlyContinue
Import-Module AiCliProfileManager -Force
Get-Module AiCliProfileManager | Format-List Path, Version
aicli version
```

或直接关闭终端重开。判断是否装到预期路径：

```
Get-Command aicli | Format-List *
```

透传给 Claude 的参数像没生效

确认参数写在 `--` 之后；在 PowerShell 中优先使用 `--%`（见上文 §6）。也可用：

```
& aicli @('start','claude-deepseek','--','--permission-mode','acceptEdits')
```

找不到目标 CLI

```
aicli doctor
codex --version
claude --version
interpreter --version
ollama --version
```

只安装你需要的上游 CLI；Open Interpreter 缺失不会影响 Codex 或 Claude Profile。

Open Interpreter 显示旧 Python 0.4.x

旧版不受支持。使用当前官方安装器：

```
irm https://www.openinterpreter.com/install.ps1 | iex
interpreter --version
```

确认输出以小写 `interpreter` 开头且版本不低于 `0.0.21`。

Claude 官方返回 401

这通常表示当前机器的 Claude Code 尚未完成官方登录，不等于 aicli 安装失败。先直接运行 `claude` 完成上游登录，再执行：

```
aicli doctor claude-official
aicli start claude-official
```

官方 Claude 意外走第三方 API

用户、项目或本地 `settings.json` 以及父终端变量可能含 `ANTHROPIC_API_KEY` 或 `ANTHROPIC_BASE_URL`：

```
aicli doctor claude-official
aicli native claude-official
```

Doctor 只报告冲突文件和字段名，不展示值。不要让 aicli 自动改写你的上游设置文件。

Qwen3.7/3.8 exact 或旧 Qwen3.7 Profile 错误

06-08 与 Qwen3.8 各自只使用独立 Workspace 按量入口；其他 Qwen3.7 Max/Plus 身份不提供入口：

```
aicli profile show codex-qwen3-7-max-paygo
aicli profile configure codex-qwen3-7-max-paygo --reuse-secret-from codex-qwen3-8-max-paygo
aicli doctor codex-qwen3-7-max-paygo
aicli profile show codex-qwen3-8-max-paygo
aicli profile configure codex-qwen3-8-max-paygo
aicli doctor codex-qwen3-8-max-paygo
```

Ollama 无法连接或模型不存在

```
ollama list
aicli doctor codex-ollama
```

公共模板使用 `127.0.0.1:11434`。确认 Ollama 正在监听且目标模型已拉取；不要把别人的端口或私有模型别名照搬到公开配置。

代理安装被拒绝

批准清单没有目标 Windows artifact 的精确 SHA256。先运行：

```
aicli proxy ccp update-check
```

发现上游版本不等于已经批准执行。普通用户不应自行绕过摘要校验。

Live Test 失败

先运行 Doctor，确认登录、Key、模型、端点和代理状态。Live Test 的失败或跳过必须保持真实状态，不能用“CLI 能打开”代替连通证据。

11. 完整命令索引

```
aicli
aicli setup
aicli version
aicli help [主题或命令]

aicli profile list [--available] [--json]
aicli profile show <Profile ID> [--json]
aicli profile configure <模板 ID> [--id <新 Profile ID>] [--reuse-existing-secret | --reuse-secret-from <Profile ID>]
aicli profile set-default <Profile ID>
aicli profile remove <Profile ID> [--yes]

aicli start <Profile ID> [--project <项目路径>] [-- <原生参数...>]
aicli run <Profile ID> --stdin --json --project <项目路径> [--sandbox-policy <policy>] [--no-web-search] [-- <原生参数...>]
aicli native <Profile ID>
aicli eject <Profile ID> [--output <新目录>]

aicli doctor [Profile ID] [--json]
aicli test <Profile ID> --live [--level text|tool|all] [--yes] [--json]

aicli proxy <ccp|cliproxy> install
aicli proxy ccp login [codex|device]
aicli proxy cliproxy login [codex|claude|device]
aicli proxy <ccp|cliproxy> logout [--purge-local-auth] [--yes]
aicli proxy <ccp|cliproxy> configure [--port <端口>|--auto-port]
aicli proxy <ccp|cliproxy> start
aicli proxy <ccp|cliproxy> stop
aicli proxy <ccp|cliproxy> status [--json]
aicli proxy <ccp|cliproxy> update-check
aicli proxy <ccp|cliproxy> update
aicli proxy <ccp|cliproxy> native

aicli update check [codex|claude|ollama|interpreter|ccp|cliproxy|self] [--json]
aicli update guide [codex|claude|ollama|interpreter|ccp|cliproxy|self]
aicli uninstall [--purge-user-data] [--yes]
```

说明：0.1.0 保留 `proxy ... update` 命令用于安全拒绝已安装实例；它不是可用的受管升级通道。未安装时请使用 `install`。

退出码：0 成功、2 用法错误、3 可用但有限制、4 不可用、5 内部错误、6 用户取消。

12. 获取帮助

```
aicli help
aicli help profile
aicli help doctor
aicli help permissions
aicli help compare
```

- CLI 会话操作: 《[Codex、Claude Code 与 Open Interpreter CLI 中文手册](#)》
- 当前兼容性状态: 《[兼容性与最终验收状态](#)》
- 安全问题: 根目录 [SECURITY.md](#)
- 隐私说明: 根目录 [PRIVACY.md](#)